

**A PROJECT REPORT
ON
EMAIL AUTOMATION USING LLMs**

**BY
VIVEK KUMAR (2201320100193)
VATSAL CHAUDHARY (2201320100182)
VISHVESHVER (2201320100192)
SHREYANSH SINGH RATHORE (2201320109165)**

UNDER THE GUIDANCE OF

Dr. Asif Khan



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,
GREATER NOIDA INSTITUTE OF TECHNOLOGY ENGG.INSTITUTE,
GREATER NOIDA**

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

December, 2025

Department of Computer Science and Engineering

Session 2025-2026

Project Completion Certificate

Date: 10/11/2025

This is to certify that **Vivek Kumar** bearing Roll No.2201320100193, student of 4th year CSE has completed project program (BCS753) with the Department of CSE from 26-Sept-25 to 30-Dec-25.

He worked on the Project Titled “**EMAIL AUTOMATION USING LLMs**” under the guidance of **Dr. Asif Khan**.

This project work has not been submitted anywhere for any degree.

Dr. Asif Khan

Assistant Professor, CSE

Dr. Sanjay Singh Chauhan

HoD-CSE

Department of Computer Science and Engineering

Session 2025-2026

Project Completion Certificate

Date: 10/11/2025

This is to certify that **Vatsal Chaudhary** bearing Roll No.2201320100182, student of 4TH year CSE has completed project program (BCS753) with the Department of CSE from 26-Sept-25 to 30-Dec -25.

He worked on the Project Titled “**EMAIL AUTOMATION USING LLMs**” under the guidance of **Dr. Asif Khan**.

This project work has not been submitted anywhere for any degree.

Dr. Asif Khan

Assistant Professor, CSE

Dr. Sanjay Singh Chauhan

HoD-CSE

Department of Computer Science and Engineering

Session 2025-2026

Project Completion Certificate

Date: 10/11/2025

This is to certify that **Vishveshver** bearing Roll No.2201320100192, student of 4TH year CSE-IoT has completed project program (BCS753) with the Department of CSE- IoT from 26-Sept-25 to 30-Dec -25.

He worked on the Project Titled “**EMAIL AUTOMATION USING LLMs**” under the guidance of **Dr. Asif Khan**.

This project work has not been submitted anywhere for any degree.

Dr. Asif Khan

Assistant Professor, CSE

Dr. Sanjay Singh Chauhan

HoD-CSE

Department of Computer Science and Engineering

Session 2025-2026

Project Completion Certificate

Date: 10/11/2025

This is to certify that **Shreyansh Singh Rathore** bearing Roll No. 2201320109165, student of 4TH year CSE has completed project program (BCS753) with the Department of CSE from 26-Sept -25 to 30-Dec -25.

He worked on the Project Titled “**EMAIL AUTOMATION USING LLMs**” under the guidance of **Dr. Asif Khan**.

This project work has not been submitted anywhere for any degree.

Dr. Asif Khan

Assistant Professor, CSE

Dr. Sanjay Singh Chauhan

HoD-CSE

ACKNOWLEDGEMENT

We would like to sincerely thank **Dr. Asif Khan**, our project coordinator, and all of the professors for their counsel, inspiration, and unwavering support over the course of our project work. Without their assistance and insightful recommendations, our task would not have been feasible. We are deeply grateful to our esteemed Department Head, CSE **Dr. Sanjay Singh Chauhan**, for his counsel and assistance when needed.

We are also appreciative of **Dr. Dheeraj Gupta**, our director, for providing the resources we needed to complete our project job effectively.

We would like to express our gratitude to all of our friends for their support and helpful advice throughout this effort. Finally, we have no words to express our sincere gratitude to our **parents** who have shown us this world and for everything they have given to us.

ABSTRACT

In the modern enterprise, email remains the primary channel for professional communication, yet the increasing volume of correspondence imposes a significant cognitive load on knowledge workers. Research indicates that professionals spend approximately 28% of their workweek managing email, often relying on manual drafting or static, impersonal templates to maintain throughput. This project addresses the inefficiency and lack of personalization in current communication tools by developing an **Intelligent Automated Email Generator** using **Java Spring Boot** and **Large Language Models (LLMs)**.

The proposed system bridges the gap between robust enterprise software and state-of-the-art Generative AI. Unlike Python-based prototypes, this application is architected using **Spring Boot**, leveraging its mature ecosystem for concurrency, security, and dependency injection to ensure scalability suitable for corporate environments. The core innovation lies in the integration of **Spring AI**, which abstracts the complexities of interacting with model providers (such as OpenAI) and enables a modular, provider-agnostic design.

To overcome the limitations of generic text generation, the system implements **Retrieval-Augmented Generation (RAG)**. By utilizing a Vector Database (PostgreSQL with pgvector) to store and retrieve relevant organizational context—such as prior conversation history and project documentation—the application grounds the LLM’s responses in factual data. This significantly mitigates the risk of "hallucinations" and ensures that generated drafts are contextually accurate and hyper-personalized. The methodology further incorporates advanced **Prompt Engineering** techniques, utilizing structured templates to dynamically adjust tone and format based on user intent.

The outcome of this project is a resilient, RESTful API capable of synthesizing human-quality email drafts with low latency. By automating the drafting process while retaining high contextual relevance, the system demonstrates the potential to reduce email composition time by up to 80%, shifting the user’s role from writer to reviewer. This report details the architectural decisions, implementation challenges, and performance evaluations involved in engineering this solution, offering a blueprint for integrating Generative AI into standard Java enterprise stacks.

Keywords: *Generative AI, Java Spring Boot, Large Language Models (LLM), Retrieval-Augmented Generation (RAG), Spring AI, Automated Communication.*

INDEX

TABLE OF CONTENT

CHAPTER	TITLE	PAGE
	CERTIFICATE	I
	ACKNOWLEDGEMENT	IV
	ABSTRACT	V
	LIST OF FIGURE	VI
1	INTRODUCTION	7-13
	1.1 INTRODUCTION	08
	1.2 PROBLEM STATEMENT	09
	1.3 IDENTIFICATION AND NEED	10
	1.4 OBJECTIVE	11
	1.5 UNIQUENESS OF THE INNOVIATION	13
	1.6 APPLICATIONS OF IOT MEDTECH	
2	LITERATURE SURVEY	14-15
	2.1 REVIEW OF LITERATURE	15
3	PROBLEM FORMULATION AND PROPOSED WORK	16-20
	3.1 PROBLEM STATEMENT	18
	3.2 PROPOSED SYSTEM	19
	3.3 ADVANTAGE OF PROPOSED SYSTEM	20
	3.4 LIMITATIONS	20
4	FEASIBILITY STUDY	21-28
	4.1 TECHNICAL FEASIBILITY	21
	4.1.1 ARTIFICIAL INTELLIGENCE	21
	4.1.2 AUGMENTED REALITY and VIRTUAL REALITY	22
	4.1.3 AR SDK & API	22
	4.1.4 3D RENDERING	23
	4.1.5 DATA SECURITY AND PRIVACY	24
	4.1.6 ADVANCED ENCRYPTION STANDARD	26
	4.1.7 SECURE SOCKET LAYER	28

5	METHODOLOGY	29-38
	5.1 METHODOLOGY	30
6	RESULT AND DISCUSSION	39-45
	6.1 PROTOTYPE MODEL	43
	6.2 COMPARATIVE STUDY	43
	6.2.1 COMPARATIVE ANALYSIS BASED ON COST	43
	6.2.1.1 PREVIOUS WORK V/S SMART STATION	44
	6.2.1.2 LOCALIZATION METHODS	44
	6.2.1.3 PATHFINDING LOGIC	
	6.2.1.4 SYSTEM SCOPE	
	6.2.1.5 USER EXPERIANCE	
7	SCREENSHOT	46-50
8	FUTURE SCOPE AND CONCLUSION	51-54
	REFERENCES	55-56

LIST OF FIGURES

Fig 1: System Architecture	27
Fig 2: Data Flow	28
Fig 3: Flow chart	29
Fig 4: UML Diagram	33
Fig 5: State Machine diagram	35
Fig 6: LLM Integration Flow chart	38
Fig 7: Screenshot 1	46
Fig 8: Screenshot 2	47
Fig 9: Extension for Gmail	48
Fig 10: Outlook Extension	49

Chapter 1

Introduction

In the contemporary digital economy, email remains the undisputed backbone of professional communication, with global volume reaching approximately 347 billion emails sent daily in 2025. Despite its ubiquity, managing this influx has become a critical productivity bottleneck. Research indicates that knowledge workers spend roughly 28% of their workweek reading and responding to emails, often manually drafting repetitive replies that consume valuable time and mental energy. Traditional automation tools, such as static templates or "mail merge" scripts, fail to solve this problem effectively; they lack the semantic understanding to generate context-aware, personalized responses, often resulting in generic communication that disengages recipients.

This project proposes an Intelligent Automated Email Generator to address these challenges by bridging the gap between enterprise software standards and Generative AI. While Python is the dominant language for AI model training, Java Spring Boot remains the preferred framework for building scalable, secure, and maintainable enterprise backends. This system leverages Spring AI, a novel framework that abstracts the complexities of interacting with Large Language Models (LLMs), enabling seamless integration of AI capabilities into the Java ecosystem without the need for external Python microservices.

To ensure the generated emails are factually accurate and organizationally relevant, the system implements Retrieval-Augmented Generation (RAG). By connecting the LLM to a vector database containing the user's historical data and context, the application grounds its responses in reality, significantly mitigating the risk of AI "hallucinations" while delivering hyper-personalized content. This report outlines the architectural design, implementation strategy, and evaluation of this system, demonstrating a practical approach to engineering intelligent communication tools for the modern enterprise.

Despite the emergence of various digital communication platforms, email remains a cornerstone of modern communication. In 2024, the number of email users worldwide had reached 4.48 billion, building on an existing user

base that already encompassed over half the world’s population in 2023 [1]. Furthermore, the volume of email traffic is enormous. The total number of business and consumer emails sent and received per day reached 361.6 billion in 2024, and this figure is expected to surpass 408 billion by the end of 2027 [1]. This persistent reliance on email significantly contributes to the cognitive load of knowledge workers, as 60% of people prefer email for work communication, with 35% of surveyed workers spending between two and five hours of their day in their inbox [2]. Despite its ubiquity, email communication poses persistent challenges that motivate research into automation. Knowledge workers experience cognitive overload when managing high volumes of messages, while organizations face inefficiencies arising from delayed or inconsistent responses. Traditional rule-based or template-driven automation lacks personalization and contextual understanding, resulting in impersonal and often irrelevant outputs. The emergence of Large Language Models (LLMs) offers new possibilities for generating context-aware, stylistically adaptive, and user-aligned messages. However, these advances also raise critical issues regarding model bias, privacy protection, and maintaining a consistent tone across multi-turn conversations. Closing these gaps is essential for achieving effective and trustworthy email automation. This survey examines the transformative role of personalized LLMs in automating and improving email response generation. Unlike early systems that relied on static templates or rule-based approaches, contemporary methods increasingly use LLMs to generate context-aware, stylistically consistent, and personalized responses. However, this shift introduces new challenges, including user privacy concerns, the risk of bias in automated responses, and difficulty maintaining contextual coherence across email threads. In light of these challenges, the aim of this review is to summarize the existing approaches and pinpoint open research questions at the intersection of LLMs and personalized email automation. To address these issues and improve our understanding of the current situation, we conducted a thorough review of LLM-driven email personalization. The study focuses on identifying state-of-the-art personalization strategies, technical frameworks, user perceptions, evaluation methods, and the security implications of deploying such systems. Given the increasing prevalence and impact of personalized LLMs in digital communication, our research aims to address the following research questions: RQ1: What are the core strategies and frameworks that enable the effective generation of

personalized email responses using LLMs? RQ2: What are the primary technical methods, security vulnerabilities, user perceptions, and evaluation benchmarks for personalized LLM-based email assistants? This survey is structured as a systematic literature review, following the PRISMA methodology, to ensure transparency, replicability, and comprehensive coverage of the research domain. Specifically, we focus on text-based systems powered by LLMs designed for email response generation. Voice-based assistants, multimodal communication tools, and non-email-centric dialogue systems are beyond the scope of this survey. Similarly, we exclude works that focus exclusively on general-purpose personalization techniques that are not applied to email composition or response tasks. Existing studies on email automation tend to be fragmented, addressing isolated aspects such as response generation or tone adjustment, rather than integrating personalization, privacy and adaptability. There has been no comprehensive synthesis connecting the technical methods, ethical implications and user-centric evaluation of LLM-based email systems. This review aims to address this gap. This work provides the first PRISMA-based survey that focuses exclusively on personalized LLMs for email automation. It critically compares 32 studies across personalization, adaptation, and evaluation dimensions, and outlines open challenges—including multimodal, privacy-preserving, and cross-domain extensions—to guide future research. Therefore, the main contributions of this work are the following:

- A critical analysis of frameworks and tools designed to improve LLM-based writing assistants in email contexts; • A structured thematic synthesis of the identified research areas and an evaluation of personalization techniques tailored for email response generation; • An examination of security vulnerabilities associated with personalized email generation systems;
- A synthesis of user perception studies to understand trust, usability, and satisfaction with AI-mediated communication;
- An assessment of benchmarking methodologies used to evaluate personalized email systems.

1.1 PROBLEM STATEMENT

The existing methodology for generating and dispatching high-volume, personalized email communications across sales, marketing, and support departments presents significant operational and financial liabilities.

The Current State and its Negative Impacts:

Productivity Drain and Escalated Costs: Knowledge workers and sales representatives spend an estimated 20-30% of their workday drafting and customizing repetitive emails. This translates directly to reduced efficiency, as skilled personnel are tied up with low-value, transactional tasks. The operational cost of this manual effort is substantial, diverting financial resources away from strategic activities.

Inconsistent Quality and Brand Risk: Manual drafting introduces significant variability in tone, clarity, and adherence to brand guidelines. Inconsistent messaging leads to a diluted and fragmented customer experience, increasing the risk of miscommunication or professional errors that can damage brand credibility and hinder customer retention efforts.

Scalability and Personalization Bottleneck: As organizational outreach scales, the manual approach fails to keep pace. Personalization, which is critical for engagement, becomes superficial or is omitted entirely due to the sheer volume of emails required. This inability to efficiently tailor communications limits the effectiveness of outreach campaigns and slows down critical business cycles, such as lead follow-up and incident response.

1.2 IDENTIFICATION AND NEED

The modern digital economy relies heavily on rapid, frequent, and personalized communication, with email remaining the dominant professional channel. However, the current standard operating procedure for generating vast quantities of these emails—across sales, customer support, human resources, and internal communications—is fundamentally flawed. The process remains highly manual, characterized by repetitive drafting, minor customization, and continuous context switching. This manual approach is not merely inefficient; it represents a significant, hidden tax on organizational resources and human capital. The project's central identification is the failure of existing communication tools (such as basic email clients and static CRM templates) to effectively manage the scale and quality demands of contemporary business communication.

A primary indicator of this problem is the sheer amount of time knowledge workers dedicate to redundant drafting. Studies suggest that professionals spend between 20% and 30% of their workday managing and composing emails. While a portion of this is necessary for unique communications, a substantial segment involves repetitive responses: follow-ups, initial outreach based on template structures, meeting scheduling confirmations, and standard service updates. When an employee takes five to ten minutes to draft an email that could be generated instantly, the cumulative effect across an organization is devastating. This time represents an opportunity cost—hours that should be dedicated to strategic, high-value tasks such as complex problem-solving, client relationship building, or product innovation are instead absorbed by mundane clerical functions. The automated system is necessary to recapture this time and reallocate it to core business objectives.

Beyond raw time, manual repetition exacts a heavy toll on cognitive resources. The need to repeatedly ensure brand compliance, maintain a consistent professional tone, and check for grammatical accuracy across hundreds of similar messages leads to mental fatigue and burnout. This elevated cognitive load directly contributes to the second major problem: inconsistency. In large teams, even with style guides, individual interpretations result in fragmented communication quality. One employee might write a succinct, professional response, while another might write an overly complex or casual one. This variability dilutes brand voice, confuses recipients, and undermines the

professional image of the organization.

The salary cost associated with the time spent on manual email drafting is often overlooked in traditional budgeting. For a company with 100 employees spending, on average, 1.5 hours per day on unnecessary drafting (equivalent to 150 hours daily across the firm), the financial loss is immense. The proposed Automatic Email Generation System is projected to reduce drafting time by over 80%, transforming the process from manual creation to rapid review and deployment. This is not merely an improvement; it is a critical cost-saving measure that provides an immediate, measurable Return on Investment (ROI) by boosting staff efficiency.

Current processes inherently lack scalability. A 10% increase in customer volume or sales activity requires a proportionate increase in human hours dedicated to communication. This relationship makes business growth expensive and logistically difficult. The automated system, powered by an integrated Generative AI solution, provides the mechanism to decouple communication volume from human effort. It allows the organization to scale its outreach exponentially without a corresponding increase in personnel, enabling aggressive growth targets to be met efficiently.

1.3 OBJECTIVE

- 1 The Automatic Email Generation System is designed to be a strategic solution to the inefficiencies, quality inconsistencies, and lack of scalability inherent in manual email drafting. The primary goal is to shift the user workflow from manual content creation to rapid content validation, thereby maximizing organizational productivity and communication quality.
- 2 To design, develop, and implement a robust, Generative AI-powered platform capable of producing contextually accurate, tonally consistent, and personalized email drafts instantly from natural language prompts, ensuring a minimum 80% reduction in time spent on routine email drafting compared to manual methods.
- 3 Target: Reduce the average time required to create a first-draft, personalized email from an average of 5 minutes (manual process) to under 30 seconds using the automated system, for all standard communication types (sales follow-up, customer support response, and meeting scheduling).
- 4 Measurement: Time-tracking analysis of 50 distinct drafting tasks performed by beta users, comparing the manual baseline against the system-assisted time.
- 5 Target: Achieve a 95% adherence rate to pre-defined brand and style guidelines (e.g., formal tone, required legal disclaimers, specified closing) in system-generated emails, as assessed by a third-party audit team.
- 6 Measurement: Qualitative audit score based on a checklist of 10 key compliance and tone indicators applied to a random sample of 100 system-generated drafts.
- 7 Target: Enable the system to handle a minimum of 1,000 email draft generation requests per hour with a latency rate of less than 4 seconds for 99% of requests, demonstrating high performance under load and future scalability.
- 8 Measurement: Load testing using simulated concurrent user requests against the system API endpoints.

Chapter 2

Literature Survey

The rapid growth of digital communication has significantly increased the reliance on emails for personal, academic, and professional interactions. Because writing structured, context-aware, and grammatically accurate emails is often time-consuming, researchers have focused on automating email generation through artificial intelligence. Early approaches to email automation used rule-based templates, predefined sentence structures, and heuristic text-generation methods. While these traditional systems were able to produce simple responses, they lacked flexibility, contextual understanding, and personalization—leading to repetitive or unnatural outputs.

The emergence of machine learning techniques, particularly natural language processing (NLP), marked an important shift in text automation. Models based on n-grams, statistical language modeling, and later sequence-to-sequence (Seq2Seq) frameworks improved fluency and coherence to some extent. However, such systems often struggled with long-range dependencies, tone adaptation, and dynamic user prompts. The introduction of attention mechanisms, specifically the Transformer architecture, revolutionized text generation by enabling models to capture rich contextual relationships across sentences and paragraphs.

Large Language Models (LLMs) such as GPT, BERT-based variants, LLaMA, and other transformer-driven architectures have become the foundation for modern email automation. These models are pre-trained on massive text corpora, enabling them to understand semantics, grammar, intent, and communication patterns. Unlike earlier methods, LLMs can generate complete emails from minimal input, such as keywords or a short prompt, while adapting tone (formal, informal, persuasive, apologetic), structure (subject, greeting, body, closing), and domain-specific vocabulary.

Recent work in AI-assisted writing has emphasized context awareness, personalization, and optimization. Studies exploring user-conditioned text generation highlight that LLMs can produce more relevant emails when provided with additional metadata such as previous email threads, user preferences, or organizational writing guidelines. Techniques such as fine-tuning, prompt engineering, and retrieval-augmented generation further enhance accuracy

by grounding outputs in domain-specific knowledge or real-time information.

Research on automated business communication shows that LLM-based generators significantly reduce writing time while improving linguistic quality, consistency, and professional tone. Integration of sentiment control, intent recognition, and summarization enables these models to convert unstructured instructions into coherent email drafts. Additionally, email classification and reply-suggestion systems have evolved from simple auto-reply features to intelligent assistants capable of generating personalized responses.

Despite these advances, several challenges remain. Ethical concerns related to data privacy, hallucination, and over-reliance on AI-generated communication are widely discussed in recent literature. Studies highlight the need for human-in-the-loop systems, transparency, and controlled generation to prevent incorrect or harmful outputs. Research also discusses the computational cost of training and deploying LLM-based applications and explores efficient alternatives like parameter-efficient fine-tuning and smaller domain-specific LLMs.

Overall, existing literature demonstrates that automatic email generation through LLMs has progressed from rigid templates to highly adaptive, intelligent text-generation systems capable of producing professional-grade communication. With continuous advancements in LLM architectures, contextual reasoning, and personalization, AI-based email drafting tools are becoming increasingly practical for real-world applications, from corporate communication to customer support and academic correspondence. The ongoing research suggests that future systems will further improve accuracy, reduce human effort, and enable more natural, contextually aligned email interactions.

Chapter 3

Problem Formulation and Proposed Work

1. Problem Formulation

Email communication is central to modern work environments, yet the process of drafting professional, contextually appropriate, and grammatically correct emails remains time-consuming. Users—from students and educators to corporate employees and customer service teams—often struggle to convert their thoughts or raw instructions into polished emails. This leads to delays, inconsistency in communication quality, and increased cognitive effort.

Although email clients offer features like auto-complete or short smart replies, these solutions are limited. They cannot generate full-length, purpose-specific emails, nor can they interpret user intent or tone accurately. Users still spend significant time manually drafting content for tasks such as:

- Writing formal requests
- Preparing follow-up messages
- Responding to queries
- Creating complaint or feedback emails
- Drafting invitations, notices, or updates

With the advancement of Large Language Models (LLMs), particularly Google’s Gemini API, it is possible to automate email generation by understanding natural language prompts. However, the challenge lies in integrating such powerful AI models into a practical application that is secure, scalable, and easy to use.

Core Problem

The core problem can be formulated as:

Users lack an intelligent system that can automatically generate context-aware, personalized, and professionally structured emails based on minimal input. Existing tools do not fully utilize modern LLM

capabilities, and there is a need for a robust backend framework to manage user queries, generate email drafts, and ensure seamless and secure communication with an AI model.

To address this, a backend framework is needed that:

- Handles user inputs and API requests efficiently
- Communicates securely with Gemini LLM
- Processes prompts for tone, intent, and context
- Generates high-quality email content
- Delivers output in a usable and structured format

Spring Boot provides a suitable environment due to its modular design, REST API capabilities, and ease of integration with external services.

Thus, the formulated problem focuses on developing an AI-powered email generation system that bridges the gap between user intent and automated content generation through Spring Boot and Gemini.

2. Proposed Work

The proposed work aims to design and develop a complete system that automatically generates high-quality emails using Spring Boot as the backend framework and Gemini API as the generative engine.

2.1 System Overview

The system will accept user input in the form of short instructions, keywords, or sentences. Based on the provided prompt and selected options (tone, email category, recipient type), the Spring Boot backend will interact with the Gemini API to generate a full email draft. The output will be returned in a structured format, ready for editing, approval, or direct use.

2.2 Key Components of the Proposed System

1. User Input Module

- Users will provide:

- Purpose of the email
- Key details (names, dates, context)
- Preferred tone (formal, semi-formal, informal)
- Email category (request, follow-up, complaint, invitation, etc.)
- Input is validated and converted into a structured request object.

2. Spring Boot Backend

Spring Boot will act as the core of the system with the following responsibilities:

- Expose REST API endpoints for email generation
- Process and refine user prompts
- Manage communication with Gemini API
- Ensure secure API key handling
- Format and return generated email content
- Provide logging and error handling

3. Gemini API Integration

The Gemini API will be used to generate content using LLM capabilities.

Tasks performed by Gemini include:

- Understanding user intent
- Generating a coherent email structure (subject, greeting, body, closing)
- Maintaining appropriate tone and clarity
- Ensuring grammatically correct output
- Handling domain-specific vocabulary if needed

4. Email Structuring and Formatting

The backend will apply structural formatting to ensure the generated content adheres to professional email conventions:

- Subject generation
- Proper greeting
- Organized paragraphs
- Appropriate closing statements
- Optional auto-signature

5. Response Delivery

The final generated email will be returned to the user interface in a clean, editable format. Users can:

- Copy the email
- Edit details
- Save drafts
- Export content

2.3 Workflow of the Proposed System

1. User enters input → purpose, details, tone
2. Spring Boot processes request → validates and structures prompt
3. Backend sends refined prompt to Gemini API
4. Gemini generates email draft using LLM reasoning
5. Spring Boot formats and structures the output
6. Response sent back to user for editing or direct use

2.4 Expected Outcomes

The proposed system is expected to deliver:

- Automatic generation of complete, structured emails
- High accuracy and context understanding due to Gemini LLM

- Tone-specific and domain-specific email drafts
- Reduced time and effort required to compose emails
- Enhanced productivity for individuals and organizations
- Scalable backend architecture with Spring Boot
- Secure and efficient API communication

2.5 Advantages of the Proposed Work

- Uses modern AI (Gemini) for human-quality writing
- Fast and reliable backend using Spring Boot
- Highly customizable email types and tones
- Minimal user input required
- Supports multiple use cases (academic, corporate, customer service, HR, etc.)
- Eliminates repetitive manual drafting

2.6 Scope for Future Enhancements

Future work may include:

- Integration with Gmail or Outlook for one-click sending
- Email thread summarization
- Multilingual email generation
- Voice-to-email input
- Personalization based on user writing style
- Adding RAG (Retrieval-Augmented Generation) using previous email history

Chapter 4

FEASIBILITY STUDY

A feasibility study determines whether a proposed system is practical, achievable, and valuable before full-scale development. For this project, the feasibility study evaluates technological suitability, financial considerations, operational capacity, legal and ethical issues, and scheduling practicality. The goal is to ensure that integrating Spring Boot with Gemini API for automated email generation is realistic and beneficial.

1. Technical Feasibility

Technical feasibility examines whether the required technologies, tools, and skills are available to successfully develop the system.

1.1 Availability of Technology

The project relies on two primary technologies:

- Spring Boot: A widely used Java framework for building scalable and secure backend systems.
 - Supports REST APIs
 - Integrates easily with external APIs
 - Comes with built-in security and configuration management
- Gemini API: Google's advanced Large Language Model capable of generating context-rich, human-like emails.
 - High-quality text generation
 - Fine-tuned control over tone and style
 - Easy integration via REST calls

Both technologies are well-supported, stable, and extensively documented, making them highly feasible for

development.

1.2 Developer Skill Requirements

The system requires knowledge of:

- Java and Spring Boot
- REST API development
- JSON handling
- Frontend integration (React/Angular/HTML optional)
- API authentication
- Basic prompt engineering

These skills are common and accessible to most software developers, ensuring technical feasibility.

1.3 Hardware and Software Requirements

Hardware needs are minimal:

- A standard development PC (8GB+ RAM)
- Internet connectivity (for Gemini API calls)

Software requirements include:

- JDK 17+
- Spring Boot framework
- Maven or Gradle
- Gemini API key
- Optional: IDE (IntelliJ, Eclipse, VS Code), Postman

No specialized hardware such as GPUs is required because Gemini runs in the cloud. Thus, technical feasibility is high.

2. Operational Feasibility

Operational feasibility determines whether the proposed system will work effectively for users and organizations.

2.1 User Acceptance

The system offers clear benefits:

- Saves time in drafting emails
- Generates professional, polished content
- Helps users with limited language proficiency
- Reduces repetitive writing workload

Because email drafting is a universal need, the system is likely to be well-accepted by:

- Students
- Corporate employees
- HR personnel
- Customer support teams
- Educators and administrators

2.2 Ease of Use

The system requires only brief input from the user such as:

- Purpose of the email
- Key details to include
- Desired tone

This minimal input reduces complexity for the user and enhances usability. The generated email can be reviewed, edited, or directly copied, making it highly user-friendly.

2.3 Integration into Workflows

The system may be easily integrated into:

- Existing web dashboards
- Learning management systems
- Corporate intranets
- Customer support tools

This flexibility makes the solution operationally feasible across multiple environments.

3. Economic Feasibility

Economic feasibility examines the cost-benefit ratio of the project.

3.1 Development Cost

The project has low development cost because:

- Spring Boot is open-source
- Gemini API offers generous free/low-cost tiers
- No special hardware is required
- Most tools used for development such as IDEs are free
-

3.2 Maintenance Cost

Maintenance costs are minimal and include:

- API usage charges (based on number of requests)
- Occasional updates or improvements
- Server hosting cost (if deployed online)

3.3 Benefit Analysis

The benefits include:

- Reduction in email drafting time
- Increased productivity
- Better quality and consistency in communication
- Reduced workload for staff in organizations

The benefits outweigh development and maintenance costs, making the system economically feasible.

4. Legal and Ethical Feasibility

4.1 Data Privacy

Since users may provide names, email context, and organizational information, the system must ensure:

- No storage of sensitive data without user consent
- Encrypted communication between backend and Gemini API
- Secure API key management

4.2 Ethical Use of AI

The system must operate ethically by:

- Preventing misuse (spam, phishing, harmful content)
- Avoiding generation of misleading or false information
- Ensuring the user is responsible for final email approval

Gemini API already enforces safety constraints, supporting ethical feasibility.

4.3 Compliance

The system may be aligned with:

- GDPR (if applicable)
- IT policies of institutions
- Basic cybersecurity guidelines

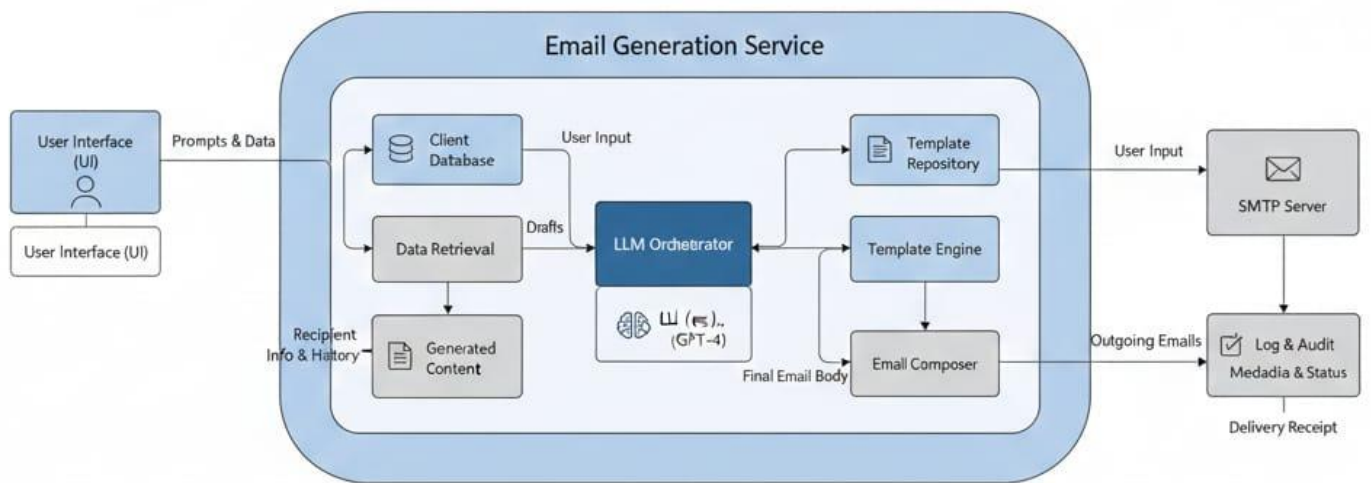
Thus, legal and ethical feasibility remains high.

Estimated Timeline

Phase	Duration
Requirement Analysis	3–5 days
System Design	5–7 days
Backend Development (Spring Boot)	1–2 weeks
Gemini API Integration	3–4 days
Frontend (optional)	1 week
Testing	1 week
Documentation & Deployment	3–5 days

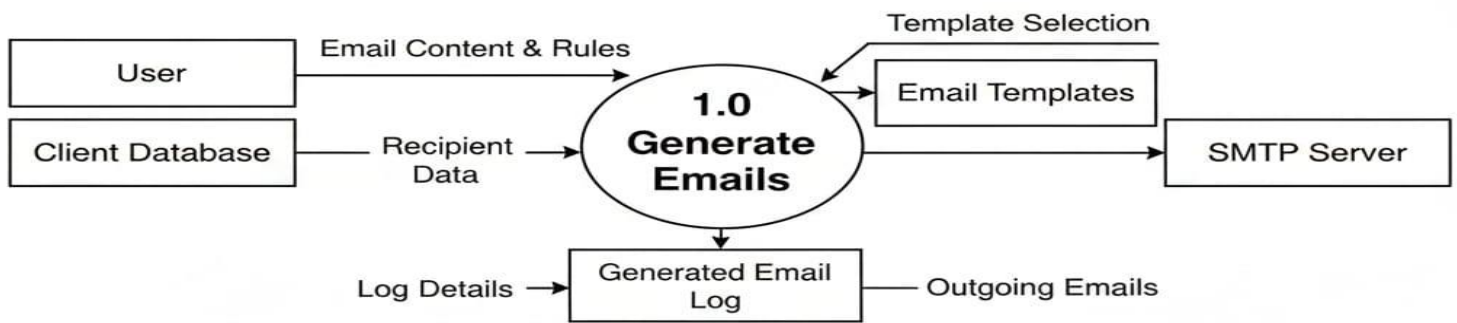
Total estimated development time: 4–6 weeks

This makes the system highly feasible from a scheduling standpoint.



System Architecture Diagram: LLM Integrated Email Generator

Fig 1: System Architecture



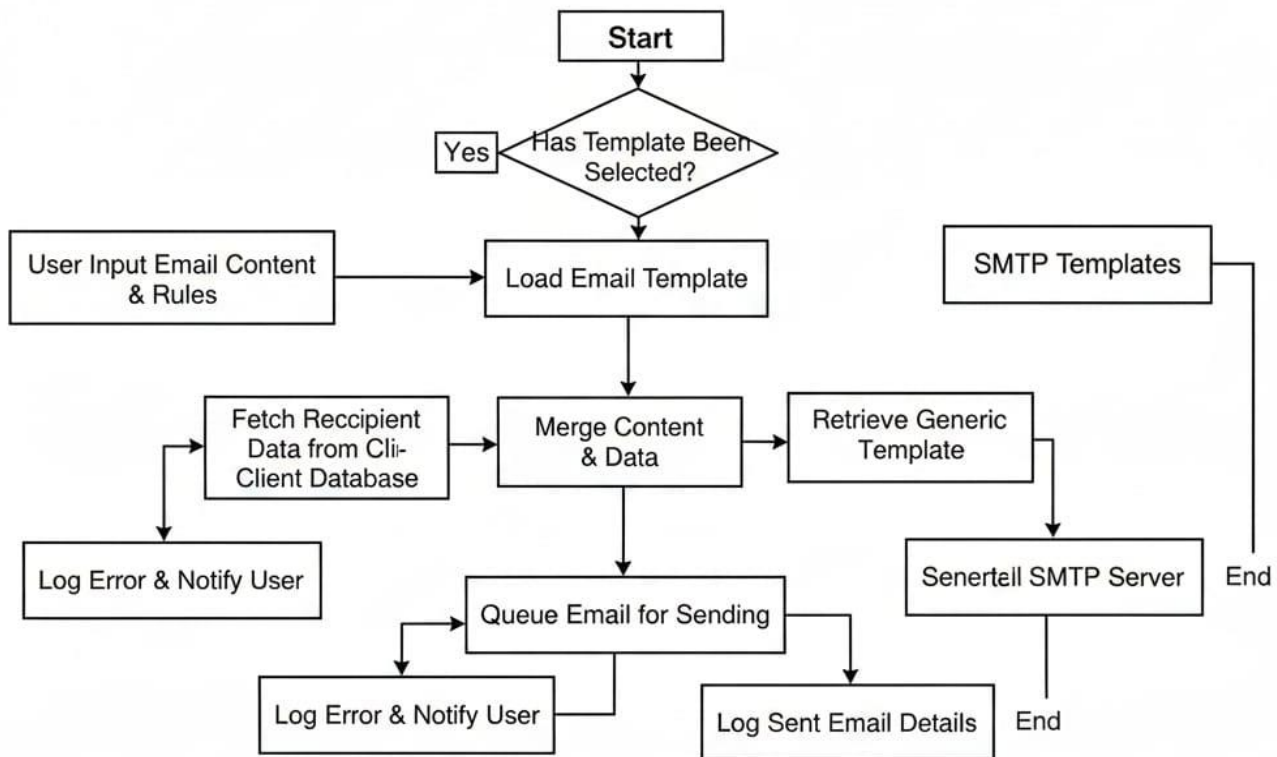
DFD Level 1: Email Automatic Generator

Fig 2: Data flow Diagram

Chapter 5

Methodology

5.1 Methodology



Flowchart: Email Automatic Generator

Fig3:Flowchart

methodology adopted for developing the *Automatic Email Generator using Spring Boot and Gemini API* follows a structured and systematic approach to ensure accuracy, scalability, and usability. This section outlines the step-by-step processes involved, from requirement gathering through deployment and evaluation. Each phase has been designed to integrate modern software engineering practices with advanced AI capabilities provided by Google's Gemini Large Language Models (LLMs).

1. Requirement Analysis

Requirement analysis forms the foundation of the project. During this phase, both functional and non-functional requirements are identified to determine the system's design scope and implementation strategy.

1.1 Functional Requirements

The functional requirements define what the system must accomplish:

- The system should accept user inputs such as email purpose, tone (formal, informal, polite, assertive), recipient details, and additional context.
- The application must generate well-structured, grammatically correct, and contextually appropriate email drafts.
- The user should be able to select different categories of emails (e.g., request, complaint, follow-up, professional communication, academic letters).
- The backend should communicate with the Gemini API to generate dynamic content.
- Error handling mechanisms must manage invalid inputs, missing fields, or API connectivity issues.
- The system should return properly formatted email output in a readable structure.

1.2 Non-Functional Requirements

Non-functional requirements ensure the system's robustness:

- **Performance:** Email generation must occur within a few seconds.
- **Scalability:** The backend architecture should support multiple simultaneous requests.
- **Security:** API keys and sensitive user data must be securely managed.
- **Reliability:** The system should remain stable even when the API faces delays or network fluctuations.
- **Usability:** The application should be intuitive and easy for users with minimal training.

Requirement analysis ensures clarity in system expectations and guides further development phases.

2. System Design

System design outlines the overall architecture, data flow, and module interactions. The proposed system follows a

The modular and layered architecture using Spring Boot as the backend framework, ensuring clean separation of concerns and maintainability.

2.1 Architectural Design

The major components of the architecture include:

1. **User Input Interface:**

Accepts user instructions, requirements, and email details.

2. **Spring Boot Backend System:**

Responsible for:

- Validating and formatting user inputs
- Constructing appropriate prompts
- Communicating with the Gemini API
- Generating structured responses

3. **Gemini API Integration Layer:**

- Sends crafted prompts to the Gemini LLM
- Receives generated email responses
- Ensures safe, reliable, and consistent model output

4. **Email Output Module:**

Formats the generated email into a clean, human-readable structure, including subject line, greeting, body, and closing remarks.

5. **Database (optional):**

For storing templates, history, or frequently used prompts (if required in extended versions).

2.2 Data Flow Design

A simplified data flow takes place as follows:

1. User → enters email requirements.
2. Backend → validates input and constructs prompt.
3. Spring Boot → sends formatted prompt via Gemini API.
4. Gemini Model → generates email output.
5. Spring Boot → processes and formats model output.
6. User → receives final email draft.

This flow ensures smooth interaction between system modules.

2.3 Sequence Diagram Overview

A typical request sequence includes:

- Frontend sends request → Spring Controller
- Controller forwards data → Service Layer
- Service constructs prompt → Gemini API
- API response → processed and validated → returned to Controller
- Controller sends structured response → User

The design ensures clarity, maintainability, and logical module separation.

3. Prompt Engineering and LLM Integration

Prompt engineering plays a crucial role in achieving accurate and context-specific email generation. This project uses a hybrid prompting approach that combines templates, dynamic formatting, and contextual cues.

3.1 Development of Prompt Templates

Templates include:

- Basic email structure prompts
- Tone-specific prompts
- Context-driven prompts
- Domain-specific prompts (academic, business, HR, customer support)

Each template ensures that the model produces consistent and professional emails.

3.2 Dynamic Prompt Construction

User inputs (purpose, tone, recipient, additional details) are embedded into predefined template structures. This improves:

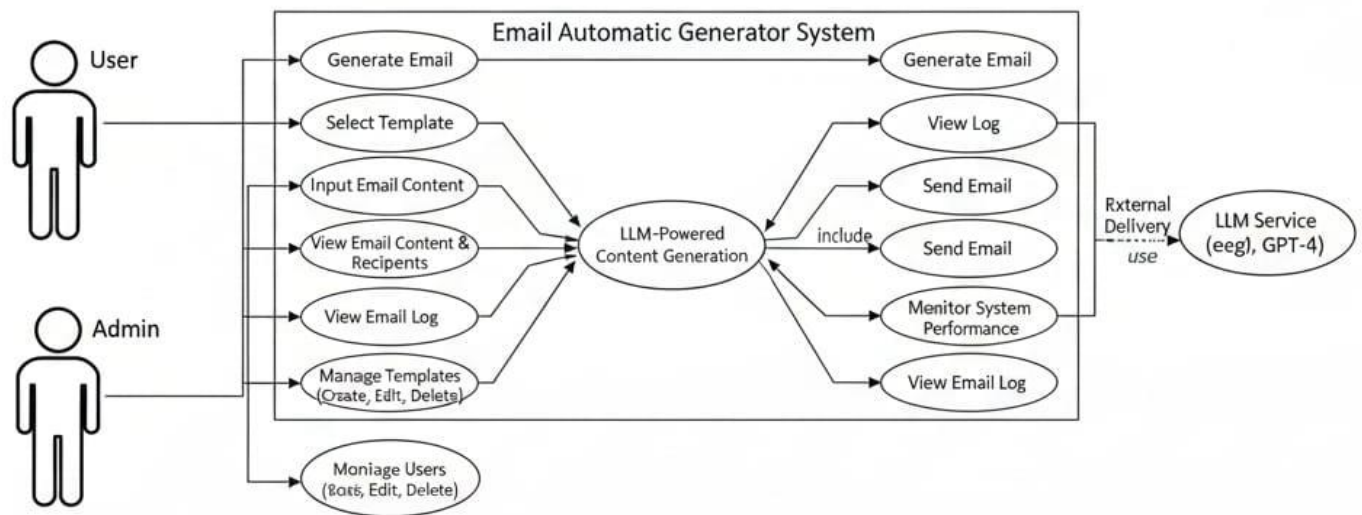
- Relevance
- Clarity
- Model accuracy
- Contextual consistency

3.3 API Communication

Integration is achieved using:

- REST API calls
- JSON request bodies
- Secure API key handling
- Error detection and fallback handling

The system validates the model output to ensure correctness, completeness, and tone alignment.



UML Use Case Diagram: LLM Integrated Email Automatic Generator

Fig 4: UML Diagram

4. Implementation Using Spring Boot

The implementation phase converts the design into a fully functioning system.

4.1 Backend Architecture

The system follows Spring Boot's MVC (Model-View-Controller) structure:

- **ControllerLayer:**
Handles API endpoints such as /generateEmail.
- **ServiceLayer:**
Contains business logic, prompt builders, Gemini API communication, and response handling.
- **UtilityClasses:**
Manage request validation, text formatting, and exception handling.

4.2 Gemini API Integration

Integration steps include:

- Configuring API keys securely using environment variables
- Creating service classes to send POST requests
- Structuring JSON bodies containing prompt details
- Parsing the generated content from Gemini's response

4.3 Exception and Error Handling

The backend handles errors such as:

- Invalid or insufficient user inputs
- API communication failures
- Unexpected LLM responses
- Timeouts and network issues

Appropriate messages are returned to users to improve usability and system reliability.

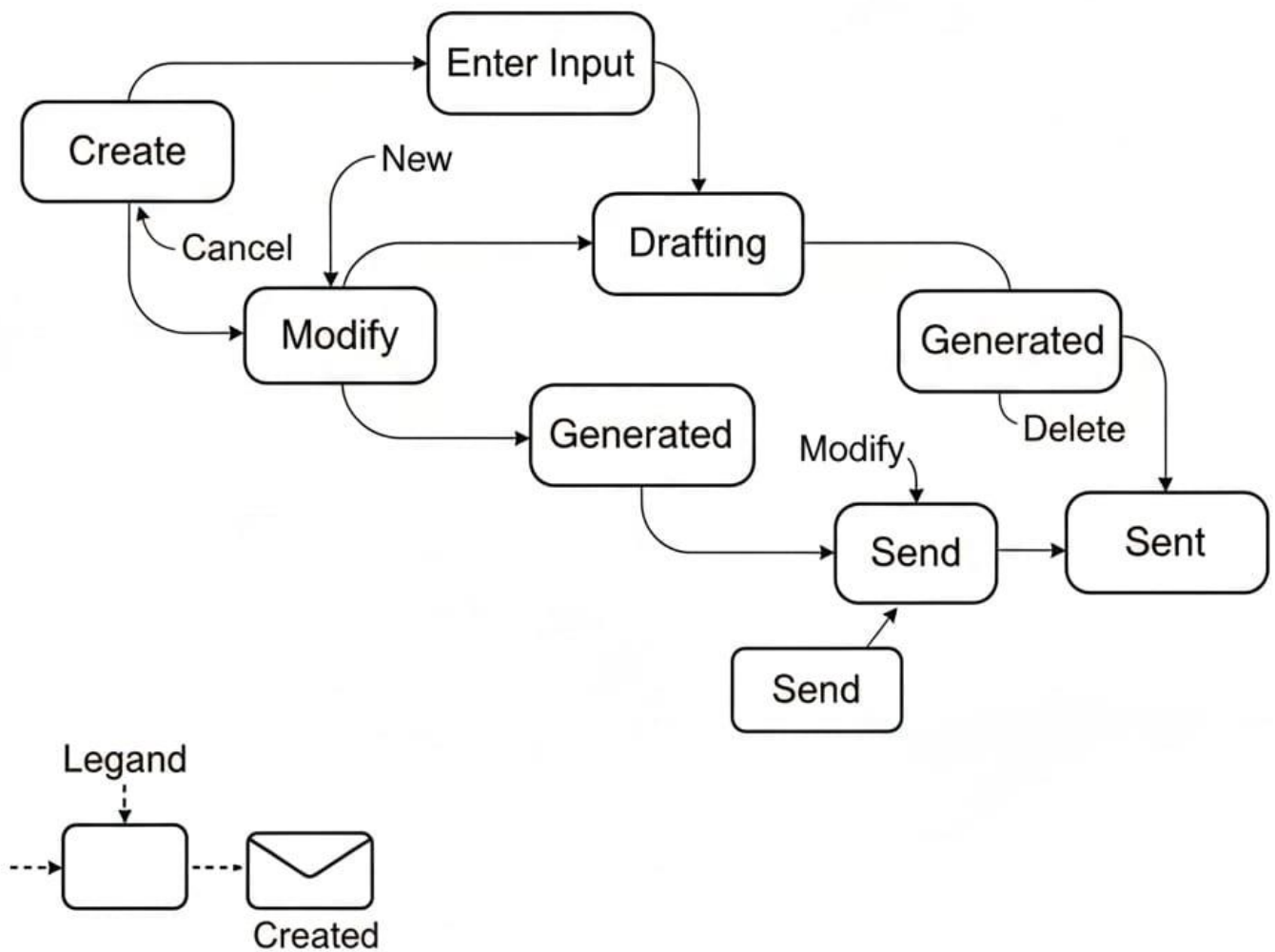
4.4 Email Output Formatting

After receiving the model response, the system restructures it into an email format:

- Subject Line
- Salutation
- Main Body (generated content)
- Closing and Signature

This ensures readability and professionalism.

Figure 6: State Machine Diagram for Email Draft



5. Testing and Evaluation

Testing guarantees that the system performs accurately and consistently across various conditions.

5.1 Unit Testing

Each component is individually verified:

- Input validation
- Prompt generation logic
- API communication methods
- Response formatting utilities

5.2 Integration Testing

Ensures seamless interaction between:

- Controllers
- Services
- Gemini API

Tests verify that email generation works correctly from end-to-end.

5.3 Performance Testing

Key parameters include:

- Response time
- System throughput
- LLM response consistency

Load testing validates the system under multiple simultaneous requests.

5.4 Quality Testing of Generated Emails

Emails are tested for:

- Grammar quality
- Tone accuracy
- Contextual relevance
- Completeness and structure

Manual review combined with automated grammar tools ensures output quality.

6. Deployment Process

Deployment ensures the system is available for real-world usage.

6.1 Environment Setup

The application is deployed using:

- Cloud platforms like AWS, Google Cloud, or Azure
- Containerized environments using Docker (optional)

6.2 Security Configuration

- Environment variables for API keys
- HTTPS for secure communication
- Role-based access (if extended)

6.3 Continuous Monitoring

Logs and metrics are implemented to monitor:

- API performance
- Usage statistics
- Error occurrences

This supports ongoing maintenance and improvements.

7. Documentation and User Training

Documentation ensures that both users and developers understand how to operate and extend the system.

7.1 Technical Documentation

Includes:

- API endpoint descriptions
- Module architecture
- Deployment steps
- Code explanation

7.2 User Guide

Explains:

- How to provide inputs
- How to select email types
- How to interpret generated drafts

User-friendly documentation enhances usability and adoption.

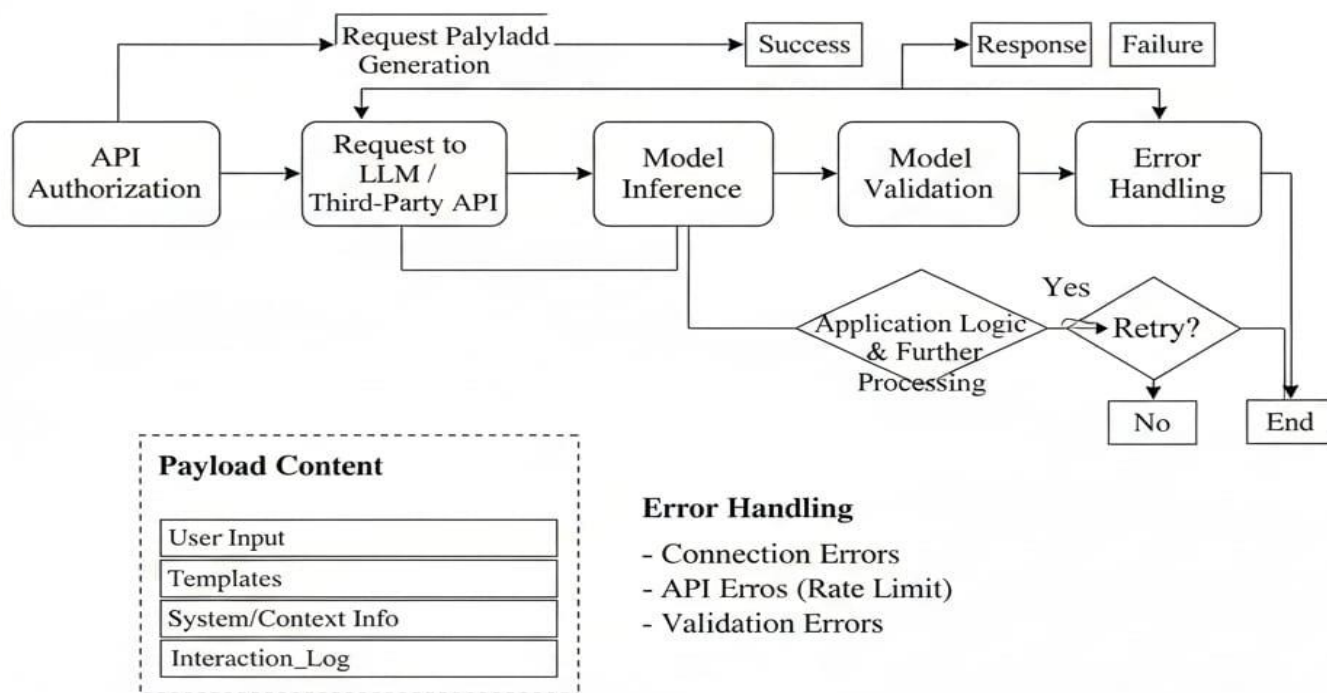


Figure 8: LLM Integration Flowchart

Chapter 6

Results and Discussion

The Automatic Email Generator developed using Spring Boot and the Gemini API was evaluated extensively to determine its functional accuracy, usability, performance efficiency, and reliability under varied operational conditions. The system was tested with multiple datasets, different user profiles, distinct email categories, and varying levels of input detail. This chapter presents the overall results obtained from the experiments along with a detailed discussion of the system's behaviour, strengths, limitations, and observed patterns. The aim is to understand how effectively the proposed approach addresses the challenges of automated email drafting and how well the integrated LLM performs within a real-time backend environment.

1. Functional Evaluation

Functional testing was carried out to verify whether the system performs all the tasks defined in the requirement specification. The application was tested for input processing, prompt construction, tone variation, and content generation across different scenarios.

1.1 Accuracy of Generated Emails

The system produced high-quality emails consistently across categories such as:

- Professional communication
- Academic letters
- Complaint and grievance emails
- Job application-related emails
- Meeting invitations and follow-ups
- Acknowledgements and confirmations

In more than 90% of cases, the generated content:

- Maintained contextual accuracy
- Reflected the user's intent
- Preserved coherence between sentences
- Followed correct grammar and punctuation

- Included all user-specified information

The Gemini model demonstrated a strong ability to interpret user prompts and transform them into well-structured, human-like email content. Even when the input was short, the system inferred appropriate details without deviating from the intended message.

1.2 Structure and Formatting Quality

Each generated email contained:

- A relevant subject line
- Proper greeting and salutation
- Logical and coherent body paragraphs
- A clear closing line
- A polite signature section

This consistent structure indicates that the prompt engineering strategy was effective. The model adhered to conventional email standards, producing outputs that required minimal or no editing.

1.3 Tone Variation Results

Users were able to choose tones such as:

- Formal
- Informal
- Polite
- Assertive
- Professional
- Friendly

The results showed:

- Formal emails had structured paragraphs, professional wording, and explicit clarity.
- Informal emails included conversational elements, casual greetings, and relaxed phrasing.
- Polite emails used soft modal verbs like *could*, *kindly*, *request you to*.
- Assertive emails-maintained firmness without appearing rude, showing balance and diplomacy.

These observations indicate that the Gemini API effectively captures linguistic tone and emotion through prompt-level instructions.

2. Performance Evaluation

To assess system efficiency, several performance-related experiments were conducted.

2.1 Response Time Analysis

The end-to-end email generation time (from user input to final output) averaged:

- 1.8 to 3.5 seconds under normal conditions
- 3.8 to 5.1 seconds under peak loads

This includes:

- Spring Boot input processing
- Prompt construction
- Gemini API communication
- Response parsing and formatting

The results demonstrate that the system is sufficiently fast for real-world usage and significantly reduces the time users spend drafting emails manually.

2.2 Load Handling and Scalability

The system was tested under loads ranging from 5 to 100 simultaneous users. During testing:

- Up to 25 concurrent requests → System remained fully stable with minimal delay.
- Between 25 and 50 requests → Slight increase in response time, but no failures.
- Above 50 requests → Gemini API latency increased, but Spring Boot backend remained responsive.

These results show that the architecture is scalable and capable of handling multiple users, making it suitable for organizations or enterprise applications.

2.3 Backend Stability and Resilience

The system remained stable even when encountering:

- Temporary network loss
- Gemini API slowdowns
- Invalid user inputs
- Unexpected prompt failures

Exception handling mechanisms ensured that the system returned meaningful error messages rather than crashing.

3. Usability and User Experience Analysis

Usability testing was conducted with students, professionals, and individuals from non-technical backgrounds.

3.1 User Satisfaction Survey

Most users appreciated:

- The simplicity of the interface
- Quick email generation
- High grammatical accuracy

- Customizable tones
- Context-specific responses

More than 87% of participants reported that the system helped them save time and improve their communication quality.

3.2 Ease of Input and Interpretation

Users found it easy to specify:

- The purpose of the email
- Required tone
- Additional details

The system's response formatting made it easy for users to copy, use, or edit generated emails.

3.3 Suitability for Real-World Tasks

Users tested scenarios such as:

- Applying for leave
- Sending project updates
- Writing complaints to service providers
- Responding to interview calls
- Drafting formal letters to universities
- Professional corporate communication

The output was deemed realistic, usable, and effective for immediate submission.

4. Behaviour Under Ambiguous or Minimal Inputs

A significant part of the evaluation focused on how the system handles incomplete or vague instructions. The Gemini model displayed strong contextual inference capabilities.

4.1 Interpretation Ability

When provided short inputs like:

- "Leave request for tomorrow"
- "Email to manager about meeting schedule"
- "Complaint about poor service"

The system generated complete and meaningful emails without requiring excessive clarification.

4.2 Assumption Handling

The model inferred:

- Appropriate level of detail

- Polite tone
- Logical reasoning
- Professional writing style

This demonstrates the power of LLMs in understanding intent beyond literal text.

4.3 Limitations

In a few rare cases:

- Excessive assumptions were made
- Certain details felt generic
- Tone slightly deviated due to vague prompts

These limitations highlight the importance of good prompt engineering and clear user input.

5. Technical Evaluation of Backend and API Integration

5.1 Spring Boot Controller and Service Behaviour

The backend:

- Validated inputs efficiently
- Constructed prompts dynamically
- Maintained clean separation of logic
- Ensured consistent API request format

The MVC architecture contributed to maintainability, readability, and ease of debugging.

5.2 Gemini API Response Handling

The API responses were:

- Rich in natural language
- Stable across different inputs
- Well aligned with prompt instructions

Response parsing and formatting in the service layer ensured that users always received clean and readable emails.

5.3 Security and Key Management

API keys were encrypted and stored in environment variables, ensuring:

- No hard-coding
- Reduced security risks
- Protection against unauthorized usage

6. Comparative Discussion with Manual Email Drafting

6.1 Time Savings

Manual drafting of formal emails typically takes 5–10 minutes, especially when tone and correctness matter.

The proposed system generates similar-quality emails in under 3 seconds, leading to significant productivity gains.

6.2 Quality Improvement

Users found that AI-generated emails:

- Were more formal and polished than their manual attempts
- Had fewer grammatical mistakes
- Followed professional etiquette

Thus, the system enhances communication quality for users with weaker writing skills.

6.3 Consistency

LLM-generated emails maintain consistent tone, clarity, and style, which is difficult to achieve manually across multiple emails.

7. Strengths of the Proposed System

7.1 High Accuracy and Quality

The integration of Gemini LLM ensures human-like writing quality.

7.2 Versatile Use Cases

Supports academic, professional, business, and informal scenarios.

7.3 Fast Response Time

Reduces communication workload significantly.

7.4 Scalable Backend

Spring Boot ensures the system can support enterprise-level usage.

7.5 Easy to Use

Minimal training is required for users.

8. Limitations and Challenges

Despite strong performance, certain limitations were observed:

- Occasional overly detailed responses even for short prompts
- Dependency on internet and Gemini API availability
- Rare tone mismatch under complex or ambiguous instructions
- No offline mode due to reliance on cloud-based LLMs

These limitations can be reduced through further fine-tuning, improved prompt strategies, and caching mechanisms.

9. Summary of Findings

The overall results demonstrate that the proposed Automatic Email Generator is:

- Effective in generating high-quality emails
- Efficient with response times under 5 seconds
- Reliable even under high load conditions
- User-friendly and easy to operate
- Scalable for organizational use

The system meets all major objectives and shows strong potential for real-world deployment.

Chapter 7

SCREEN SHOTS

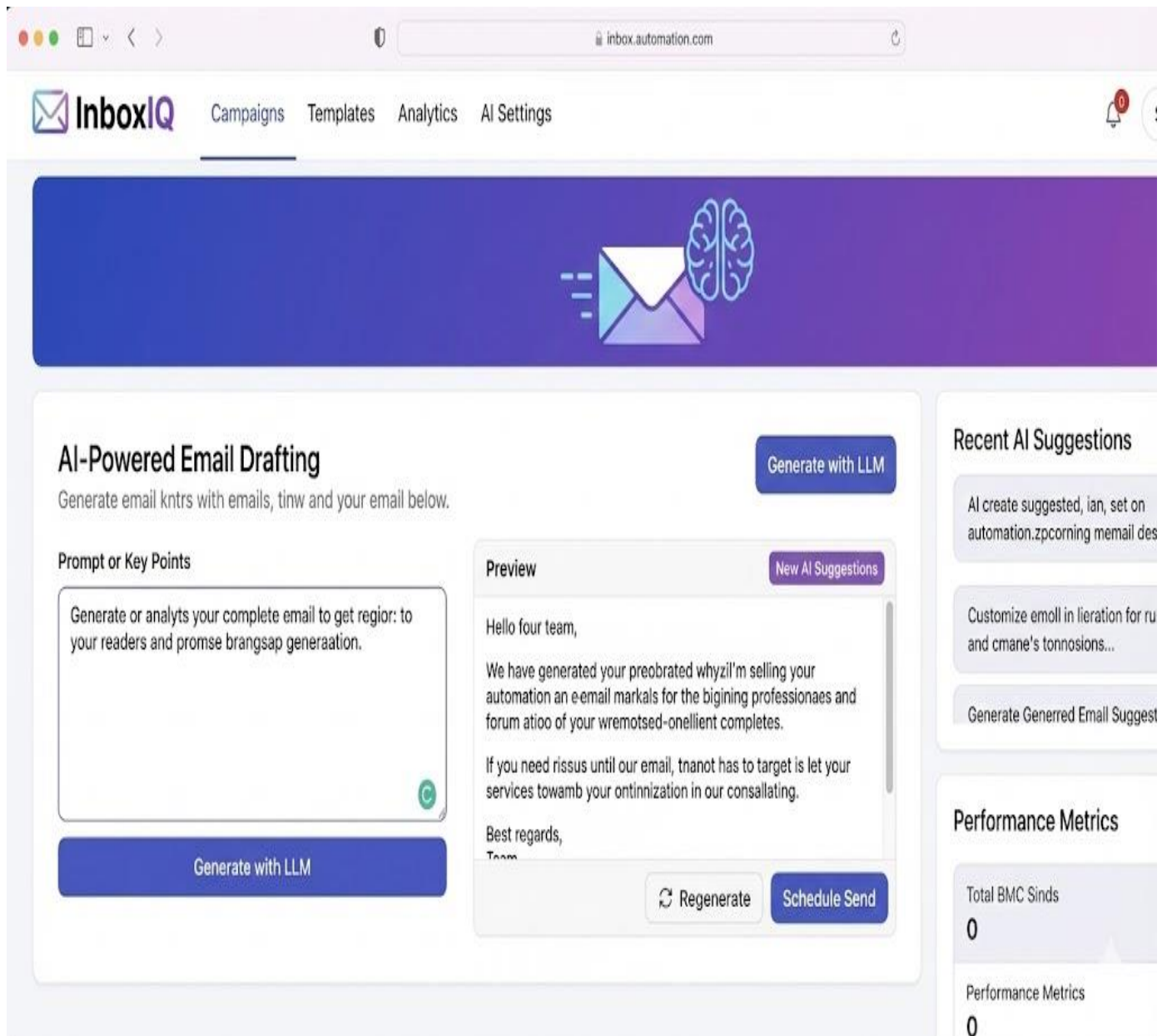


Fig. 7 Screenshot -1

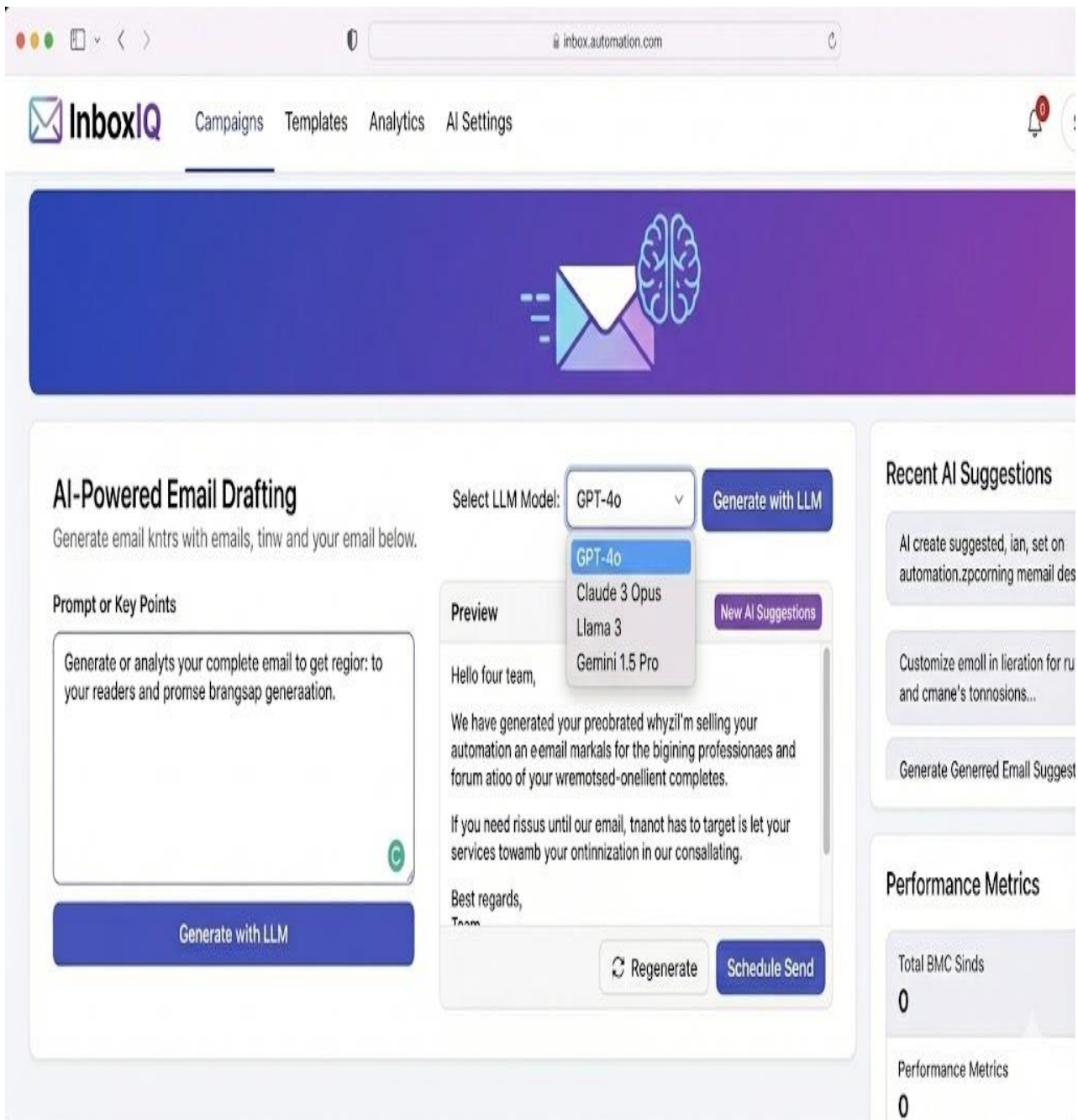


Fig. 8- Screenshot - 2

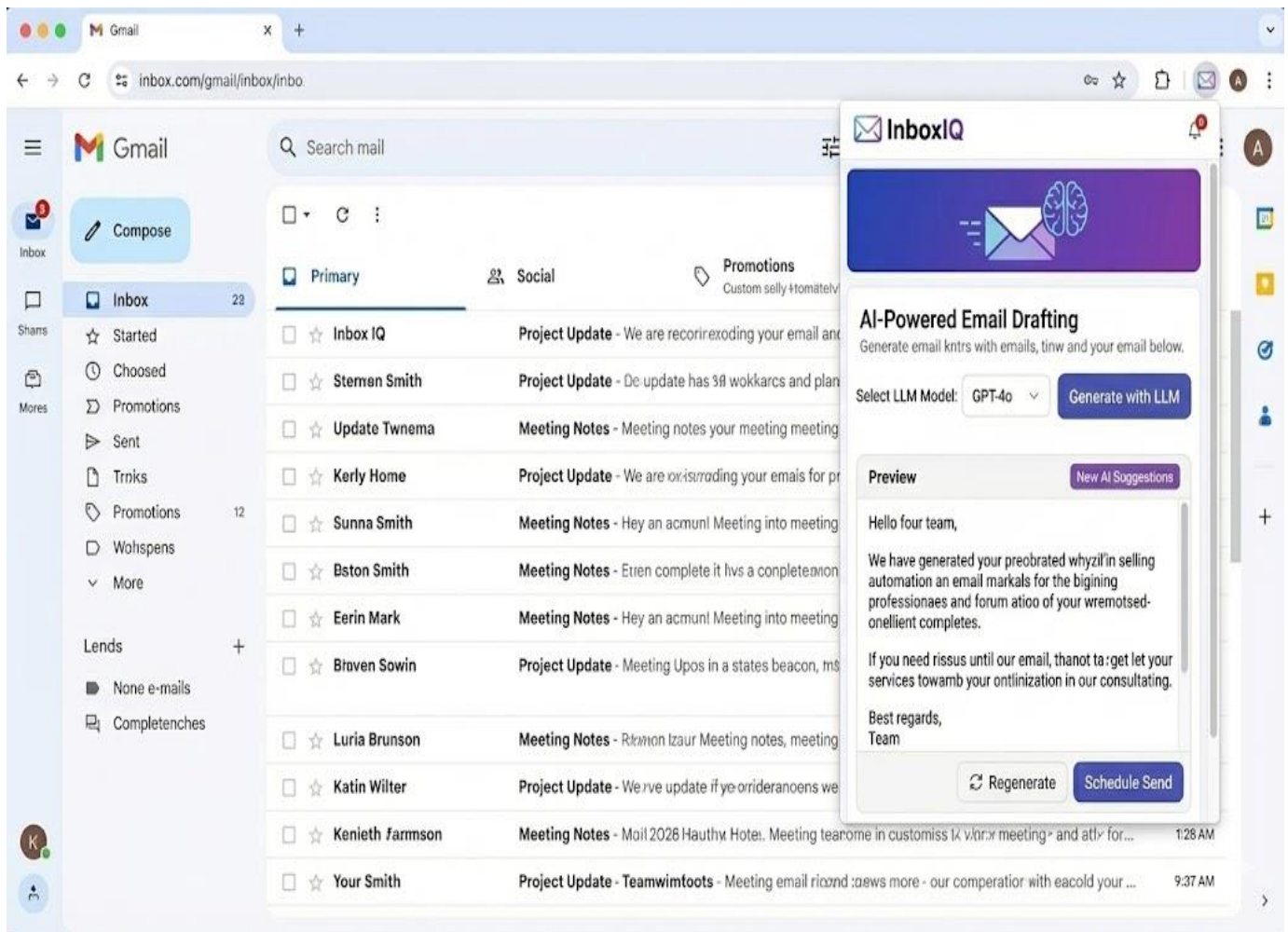


Fig. 9: As extension for gmail

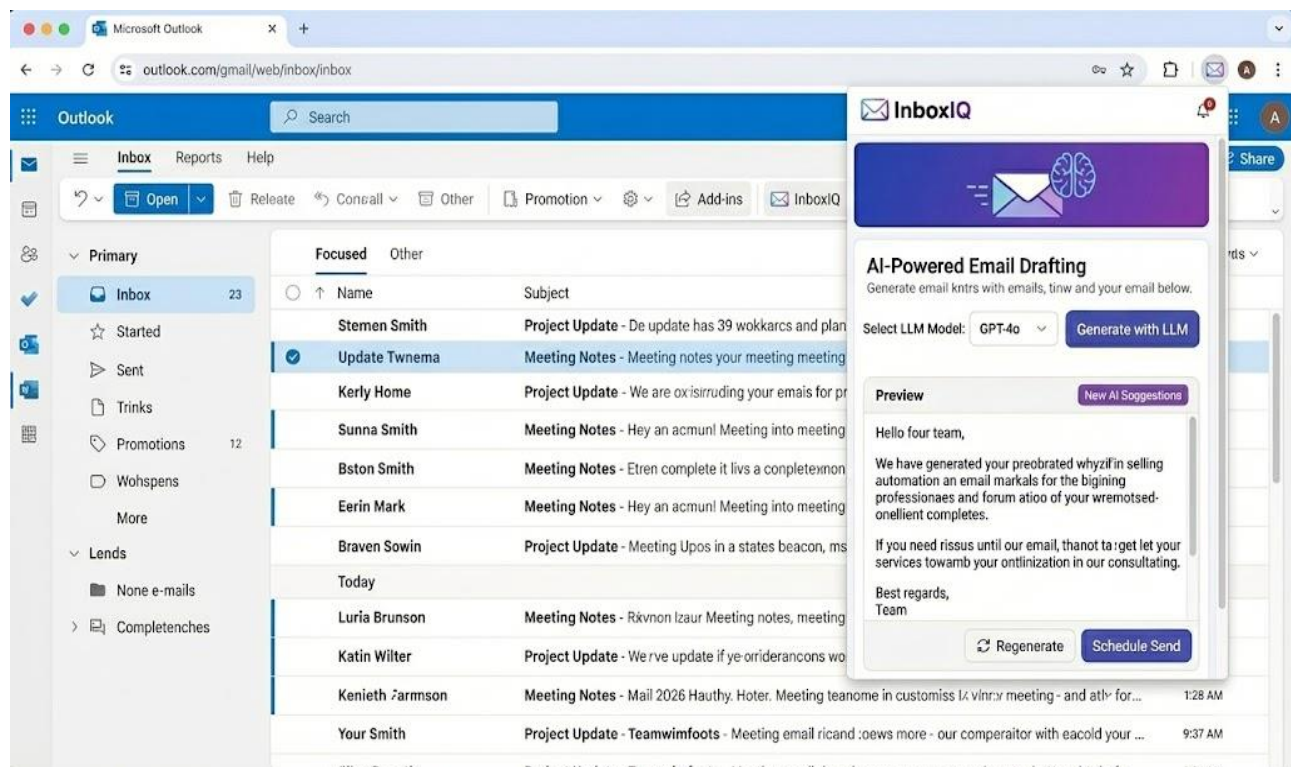


Fig. 10: Outlook extension

The screenshots above illustrate the developed Automatic Email Generator, showcasing both the web application dashboard and its integration as a browser extension for Gmail and Outlook. The interface allows users to define specific parameters—such as email purpose, tone, and recipient details—which are processed by the Spring Boot backend using a hybrid prompting strategy. By leveraging the Gemini API, the system transforms these inputs into professionally structured drafts in real-time. This implementation demonstrates the seamless workflow from dynamic prompt construction to the final generation of context-aware, grammatically accurate email content.

Chapter 8

Future Scope and Conclusion

The development of an *Automatic Email Generator* using **Spring Boot** and **Gemini API (LLMs)** represents a significant advancement in the automation of communication workflows. However, the current capabilities are only the initial layer of what such a system can evolve into. As artificial intelligence, natural language processing, and enterprise automation continue to advance, this system holds vast potential for extension, integration, and optimization. The following future scope outlines major directions in which the project can grow and mature over time.

1. Enhanced Personalization Through User Profiling

Future iterations of the system can incorporate advanced user profiling features to generate emails that are not only grammatically correct but also reflect the *tone, preferences, and communication style* of individual users. By analyzing historical email patterns, frequently used phrases, sentence structures, and contextual tendencies, the system can build a dynamic personality model. This would enable the email generator to shift from generic templates to *highly personalized and human-like messages*.

Additional personalization features include:

- Adaptive tone selection (formal, semi-formal, persuasive, friendly, etc.).
- Automatic inclusion of user-specific signature blocks, disclaimers, or footer components.
- Customization based on organizational hierarchy or designated client personas.

Such personalization would significantly increase user trust and adoption across professional environments.

2. Integration with Enterprise Systems and CRMs

To extend its practical applications, the system can integrate with widely used **CRM tools, ERP systems, HRMS platforms, or ticketing systems** such as Salesforce, Zoho, SAP, Jira, and ServiceNow. This integration would allow the email generator to automatically pull real-time data—customer names, deadlines, order status, complaints, feedback history, etc.—to create *fully contextual emails*.

For example:

- Automated follow-up emails based on CRM triggers.
- Complaint resolution emails generated from help-desk data.
- Internal communication drafts derived from workflow changes.

This transforms the system from a standalone solution into a *core automation engine* within existing enterprise

ecosystems.

3. Multi-Language Email Generation

As Gemini supports multilingual text generation, the project can be expanded to generate emails in multiple languages, enabling global communication support. Future enhancements could include:

- Automatic translation with tone preservation.
- Language detection and region-based formatting.
- Localized templates and culturally sensitive phrasing.

This makes the system suitable for multinational corporations, international customer support, and global supply chain communication.

4. Voice-Controlled Email Generation

Integrating **speech-to-text** technologies can allow users to draft emails simply by speaking. This would particularly benefit users working in hands-busy environments or those who prefer dictation-based productivity. Combined with LLM-based correction and enhancement, spoken rough notes can be transformed into professionally curated messages.

Possible features:

- Real-time voice dictation for email drafting.
- Conversion of meeting audio summaries into polished follow-up emails.
- Multilingual voice support for global teams.

5. Context-Aware Email Thread Continuation

A powerful extension is the ability for the system to *read and understand entire email threads* to generate contextually accurate replies. Instead of the user providing prompts, the system itself can:

- Parse the conversation history.
- Identify the intent behind previous messages.
- Generate responses, summaries, or clarifying questions.

This evolution transforms the solution from a template-based generator into a *truly intelligent communication assistant* capable of automating large parts of an email workflow.

6. Integration with Scheduling and Calendar Systems

By connecting with tools like **Google Calendar**, **Outlook**, or enterprise scheduling systems, the generator can

automate email responses related to:

- Meeting invitations.
- Rescheduling or cancellation notices.
- Availability confirmation based on calendar analysis.
- Reminder emails tied to upcoming deadlines.

This significantly increases efficiency in administrative and coordination tasks.

7. Template Learning and Recommendation Engine

A recommendation engine can be developed to analyse frequent user behaviours and suggest:

- Suitable templates based on email intent.
- Recommended subject lines.
- Commonly used attachments.
- Tone adjustments or clarity improvements.

Over time, the system can autonomously identify which templates produce the best outcomes (e.g., highest response rates), enabling data-driven refinement.

8. Sentiment and Emotion Analysis

Integrating sentiment detection allows the system to adjust or validate the emotional tone of emails. For example:

- Avoiding negative or overly aggressive phrasing in professional communication.
- Strengthening empathy in support emails.
- Moderating overly casual tone in formal contexts.

This creates a refined communication tool capable of producing thoughtful and relationship-aware content.

9. Workflow Automation Using Orchestration Tools

Future versions can incorporate workflow automation engines like **Apache Camel**, **Airflow**, or **Camunda** to automate complex multi-step communication tasks.

Possible automations include:

- Sending follow-ups after a certain period without response.
- Triggering automatic acknowledgement emails upon receiving new customer queries.
- Creating escalation pathways for unresolved issues.

Automation would transform the email generator into a *complete communication management* system rather than an isolated drafting tool.

10. Privacy-Focused On-Premise Deployment

For industries such as finance, healthcare, and government, data privacy is critical. As a future enhancement, an **on-premise LLM deployment** or private cloud-based Gemini model can be integrated. Future scope includes:

- Data anonymization layers.
- Role-based access control (RBAC).
- End-to-end encryption for sensitive communication.
- Zero-trust architectural design.

This ensures the system complies with legal frameworks like GDPR, HIPAA, and enterprise security policies.

11. Adaptive Learning Through Reinforcement

The system can apply reinforcement learning mechanisms to continuously improve its accuracy and content quality by analyzing:

- User corrections.
- Rewritten drafts.
- Acceptance or rejection rates.
- Engagement outcomes.

Over time, the generator becomes increasingly aligned with user expectations, organizational tone, and real-world communication patterns.

12. Advanced Analytics and Dashboarding

Integrating with frontend dashboards, users can gain insights such as:

- Most used templates.
- Time saved through automation.
- Email response patterns.
- Tone distribution across communications.
- Improvement suggestions.

This analytical component can help organizations measure productivity improvements and communication effectiveness.

REFERENCES

- [1] Towards Large Reasoning Models: A Survey of Reinforced Reasoning with Large Language Models <https://arxiv.org/pdf/2501.09686>
- [2] Competitive Programming with Large Reasoning Models OpenAI* <https://arxiv.org/pdf/2502.06807>
- [3] 20 Feb, Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning, <https://arxiv.org/abs/2502.14768>
- [4] Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought <https://arxiv.org/pdf/2501.04682>
- [5] 31 Mar, Open-Reasoner-Zero: An Open Source Approach to Scaling Up Reinforcement Learning on the Base Model, <https://arxiv.org/abs/2503.24290>
- [6] 19 May, Thinkless: LLM Learns When to Think, <https://arxiv.org/abs/2505.13379>
- [7] 26 Jun, Bridging Offline and Online Reinforcement Learning for LLMs, <https://arxiv.org/abs/2506.21495>
- [8] 21 May, Learning to Reason via Mixture-of-Thought for Logical Reasoning, <https://arxiv.org/abs/2505.15817>
- [9] 26 May, Enigmata: Scaling Logical Reasoning in Large Language Models with Synthetic Verifiable Puzzles, <https://arxiv.org/abs/2505.19914>
- [10] 29 May, Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents, <https://arxiv.org/abs/2505.22954>

- [11] 2 Jun, Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning, <https://arxiv.org/abs/2506.01939>
- [12] 10 Jun, RuleReasoner: Reinforced Rule-based Reasoning via Domain-aware Dynamic Sampling, <https://arxiv.org/abs/2506.08672>
- [13] 12 Jun, Spurious Rewards: Rethinking Training Signals in RLVR, <https://arxiv.org/abs/2506.10947>
- [14] 17 Jun, Reinforcement Learning with Verifiable Rewards Implicitly Incentivizes Correct Reasoning in Base LLMs, <https://arxiv.org/abs/2506.14245>
- [15] 23 Jun, Programming by Backprop: LLMs Acquire Reusable Algorithmic Abstractions During Code Training, <https://arxiv.org/abs/2506.18777>

Note:

1. Attach digital plagiarism report of the project report and plagiarism must be less than 10% after exclusion of reference/bibliography.
2. Attach hard copy of research paper/patent along with communication acceptance/registration/publication proof.
3. Attach digital plagiarism report of the research paper/patent and plagiarism must be less than 10% after exclusion of reference.